

A Project Report On

**AGENTVERSE - AI DRIVEN VIRTUAL COMPANY
SIMULATION USING MULTI AGENT COLLABORATION**

Submitted in partial fulfillment of the requirements for the award of the degree
of

BACHELOR OF TECHNOLOGY

In

CSE - ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Submitted By

ADABALA PUJITHASRI NAGA GANGABHAVANI	22P31A4201
MATTAPARTHI SAI SANTOSH VAMSI	22P31A4222
SOMISETTI NAGENDRA	22P31A4240
KANTIPUDI SRINIDHI	22P31A4218
DADI VENKATA JAGADESWARA KRANTHIKUMAR	22P31A4206

Under the esteemed supervision of

Dr. B. SRINIVAS, M.Tech., Ph.D.

ASSOCIATE PROFESSOR



**DEPARTMENT OF CSE - ARTIFICIAL INTELLIGENCE &
MACHINE LEARNING**

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY (A)

Permanently Affiliated to JNTUK, Kakinada * Approved by AICTE New Delhi

Accredited by NBA, Accredited by NAAC (A+) with 3.4 CGPA

Aditya Nagar, ADB Road, Surampalem, Kakinada District, Andhra Pradesh.

2022-2026

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

(An Autonomous Institution)

Permanently Affiliated to JNTUK, Kakinada * Approved by AICTE New Delhi

Accredited by NBA, Accredited by NAAC (A+) with 3.4 CGPA

Aditya Nagar, ADB Road, Surampalem, Kakinada District, Andhra Pradesh

DEPARTMENT OF CSE - ARTIFICIAL INTELLIGENCE & MACHINE LEARNING



CERTIFICATE

This is to certify that the project work entitled “**AGENTVERSE: AI DRIVEN VIRTUAL COMPANY SIMULATION USING MULTI AGENT COLLABORATION**”, is a bonafide work carried out by **ADABALA PUJITHASRI NAGA GANGABHAVANI (22P31A4201), MATTAPARTHI SAI SANTOSH VAMSI (22P31A4222), SOMISETTI NAGENDRA (22P31A4240), KANTIPUDI SRINIDHI (22P31A4218), DADI VENKATA JAGADESWARA KRANTHIKUMAR (22P31A4206)**, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY IN CSE - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING** from **ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY (A)** during the academic year **2025-2026**.

PROJECT SUPERVISOR

Dr. B. SRINIVAS, M.Tech., Ph.D.

Associate Professor

HEAD OF THE DEPARTMENT

Dr. K. NAGA BHARGAVI, M. Tech, Ph.D.

Associate Professor

EXTERNAL EXAMINER

DECLARATION

We hereby declare that this project entitled “**AGENTVERSE: AI DRIVEN VIRTUAL COMPANY SIMULATION USING MULTI AGENT COLLABORATION**”, has been undertaken by us and this work has been submitted to Department of CSE – Artificial Intelligence and Machine Learning, **Aditya College of Engineering & Technology(A)** affiliated to JNTUK, Kakinada, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in CSE - Artificial Intelligence and Machine Learning**.

We further declare that this project work has not been submitted in full or part to any other University or educational institute for the award of any degree or diploma.

PROJECT ASSOCIATES

ADABALA PUJITHASRI NAGA GANGABHAVANI	(22P31A4201)
MATTAPARTHI SAI SANTOSH VAMSI	(22P31A4222)
SOMISETTI NAGENDRA	(22P31A4240)
KANTIPUDI SRINIDHI	(22P31A4218)
DADI VENKATA JAGADESWARA KRANTHIKUMAR	(22P31A4206)

ACKNOWLEDGEMENT

It is with immense pleasure that we would like to express our indebted gratitude to our **project supervisor, Dr. B. Srinivas**, M.Tech., Ph.D., who has guided us a lot and encouraged us in every step of the project work, his valuable moral support and guidance throughout the project helped us to a great extent.

We wish to express our sincere thanks to the **Head of the Department, Dr. K. Naga Bhargavi**, M.Tech, Ph.D, for her valuable guidance given to us throughout the period of the project work and throughout the program.

We feel elated to thank **Dr. P. S.V.V. Ravi Kumar**, M.Tech,Ph.D., **Dean (Academics)** of Aditya College of Engineering & Technology (A) for his cooperation in completion of our project and throughout the program.

We express our gratitude to **Dr. A. Ramesh**, M.Tech,Ph.D., **Principal**, of Aditya College of Engineering & Technology (A) for his cooperation in completion of our project and throughout the program.

We wish to express our sincere thanks to all faculty members, lab technicians for their valuable assistance throughout the period of the project.

We avail this opportunity to express our deep sense and heartfelt thanks to the **Management of Aditya College of Engineering & Technology (A)** for providing a great support for us in completing our project and also throughout the program.

PROJECT ASSOCIATES

ADABALA PUJITHASRI NAGA	(22P31A4201)
GANGABHAVANI	
MATTAPARTHI SAI SANTOSH VAMSI	(22P31A4222)
SOMISETTI NAGENDRA	(22P31A4240)
KANTIPUDI SRINIDHI	(22P31A4218)
DADI VENKATA JAGADESWARA	(22P31A4206)
KRANTHIKUMAR	



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

An AUTONOMOUS Institution

Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade
Recognized by UGC under Sections 2(F) and 12(B) of UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.
Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

VISION

To induce higher planes of learning by imparting technical education with

- ✓ International standards
- ✓ Applied research
- ✓ Creative Ability
- ✓ Value based instruction and to emerge as a premiere institute

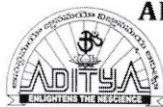
MISSION

Achieving academic excellence by providing globally acceptable technical education by forecasting technology through

- ✓ Innovative Research And development
- ✓ Industry Institute Interaction
- ✓ Empowered Manpower




Principal
PRINCIPAL
Aditya College of
Engineering & Technology
SURAMPALEM- 533 437



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY(A) (An AUTONOMOUS Institution)

Approved by AICTE, New Delhi * Permanently Affiliated to JNTUK, Kakinada
Accredited by NBA* Accredited by NAAC A+ Grade with CGPA of 3.40
Recognized by UGC Under Sections 2(f) and 12(B) of the UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Gandepalli Mandal, Kakinada District - 533437, A.P
Ph. 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

DEPARTMENT OF CSE- ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

VISION

To be a recognized Computer Science and Engineering hub striving to meet the growing needs of the Industry and Society.

MISSION

- M1:** Imparting Quality Education through state-of-the-art infrastructure with industry collaboration.
- M2:** Enhance Teaching Learning Process to disseminate knowledge.
- M3:** Organize Skill based, Industrial and Societal Events for overall Development.

K. Bheegam
Head of the Department
Head of the Department
Dept. of CSE (AI & ML, Data Science)
Aditya College of Engineering
& Technology
SURAMPALAM-533437



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY(A)
(An AUTONOMOUS Institution)

Approved by AICTE, New Delhi * Permanently Affiliated to JNTUK, Kakinada
Accredited by NBA* Accredited by NAAC A+ Grade with CGPA of 3.40
Recognized by UGC Under Sections 2(f) and 12(B) of the UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Gandepalli Mandal, Kakinada District - 533437, A.P
Ph. 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

DEPARTMENT OF CSE- ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

PROGRAM SPECIFIC OUTCOMES

PSO1: AI and ML system development

Design and develop, and implement AI and ML systems by applying fundamental principles, algorithms and techniques.

PSO2: Data-Driven Decision-Making

Collect, Preprocess and analyze large and diverse datasets to extract meaningful insights using AI and ML techniques to support data-driven decision-making processes in various domains.

K. Bheegam
Head of the Department
Head of the Department
Dept. of CSE (AI & ML, Data Science)
Aditya College of Engineering
& Technology
SURAMPALÉM-533 437



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY(A)
(An AUTONOMOUS Institution)

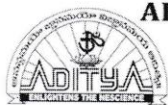
Approved by AICTE, New Delhi * Permanently Affiliated to JNTUK, Kakinada
Accredited by NBA* Accredited by NAAC A+ Grade with CGPA of 3.40
Recognized by UGC Under Sections 2(f) and 12(B) of the UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Gandepalli Mandal, Kakinada District - 533437, A.P.
Ph. 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

DEPARTMENT OF CSE- ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

PROGRAM EDUCATIONAL OBJECTIVES

- PEO1:** Engage to work productively as design and development Engineers, cater to supportive and Leadership roles in multidisciplinary domains.
- PEO2:** Learn and advance their careers by attaining professional certification and seeking higher education.
- PEO3:** Possess skill in Data Science expertise ready to serve the society locally and internationally.

K. Bhegan
Head of the Department
Head of the Department
Dept. of CSE (AI & ML, Data Science)
Aditya College of Engineering
& Technology
SURAMPALEM-533 437



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY(A)

(An AUTONOMOUS Institution)

Approved by AICTE, New Delhi * Permanently Affiliated to JNTUK, Kakinada

Accredited by NBA * Accredited by NAAC A+ Grade with CGPA of 3.40

Recognized by UGC Under Sections 2(f) and 12(B) of the UGC Act, 1956

Aditya Nagar, ADB Road, Surampalem, Gandepalli Mandal, Kakinada District - 533437, A.P.
Ph. 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

DEPARTMENT OF CSE- ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

PROGRAMME OUTCOMES

- PO1: Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- PO2: Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- PO3: Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- PO4: Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- PO5: Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- PO6: The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- PO7: Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- PO8: Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- PO9: Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- PO10: Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- PO11: Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- PO12: Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

K. Bheegani
Head of the Department
Head of the Department
Dept. of CSE (AI & ML, Data Science)
Aditya College of Engineering
& Technology
SURAMPALEM-533 437

ABSTRACT

Artificial Intelligence systems are widely used to solve user problems, but traditional systems often rely on a single model, which limits their ability to handle complex and multi-domain queries effectively. To overcome this limitation, a multi-agent approach can be used, where multiple agents work together to provide better and more accurate solutions.

The AgentVerse system is an intelligent AI-based application developed to process user queries using multiple specialized agents. The system is designed to handle different domains such as career guidance, business ideas, research assistance, and general problem-solving. It uses Large Language Models (LLMs) to understand user input and generate meaningful responses. A central system analyzes the query and assigns tasks to suitable agents based on their roles.

The system stores and retrieves relevant information to improve response quality and maintain user data effectively. It also uses a backend framework to handle user requests and ensure smooth communication between different components. The agents work together to process the query and generate outputs, which are then combined to provide a clear and structured response to the user.

The main objective of AgentVerse is to provide accurate, context-aware, and easy-to-understand solutions using multi-agent collaboration. The system aims to improve user experience and demonstrate an effective solution for solving real-world problems using advanced AI technique.

INDEX

CHAPTER	PAGE NO.
ABSTRACT	x
LIST OF FIGURES	xiv-xv
LIST OF TABLES	xvi
1. INTRODUCTION	1-5
1.1 Purpose of the System	1-2
1.2 Problem Statement	2
1.3 Objectives of Study	2-3
1.4 Scope of the System	3
1.5 Current System	4
1.6 Proposed System	4-5
2. REQUIREMENT ANALYSIS	6-9
2.1 Functional Requirements	6
2.2 Non – Functional Requirements	7-9
2.2.1 User Interface and Human Factors	7
2.2.2 Software Requirements	7
2.2.3 Hardware Requirements	7-8
2.2.4 Usability	8
2.2.5 Reliability	8
2.2.6 Performance	8
2.2.7 Supportability	8
2.2.8 Physical Environment	9

2.2.9	Security Requirements	9
2.2.10	Resource Requirements	9
3.	SYSTEM ANALYSIS	10-16
3.1	Introduction	10-12
3.2	Use Cases	12-16
3.2.1	Actors	13
3.2.2	List of use cases	13-14
3.2.3	Use case diagrams	14-16
4.	SYSTEM DESIGN	17-31
4.1	Introduction	17
4.2	System Architecture	17
4.3	System Object Model	18-19
4.3.1	Introduction	18
4.3.2	Subsystems	18-19
4.4	Object Description	19-22
4.4.1	Objects	19-21
4.4.2	Class Diagrams	21-22
4.5	Dynamic Model	22-26
4.5.1	Sequence Diagram	22-24
4.5.2	Activity Diagrams	24-26
4.6	Database Design	26-28
4.6.1	Database Tables	27-28
4.7	Static Model	28-31
4.7.1	Component Diagrams	29-30

4.7.2	Deployment Diagrams	30-31
5.	IMPLEMENTATION	32-49
5.1	Software Used	32-39
5.2	Source Code	39-49
6.	TESTING	50-53
6.1	Introduction	50
6.2	Types of Testing	51-53
6.2.1	Black Box Testing	51
6.2.2	White box Testing	51-52
6.2.3	Unit Testing	52-53
7.	OUTPUT SCREENS	54-56
	CONCLUSION	57
	FUTURE ENCHANCEMENTS	58
	BIBLIOGRAPHY	59

LIST OF FIGURES

S.No.	NAME OF FIGURE	PAGE No.
1	3.2.3.1 New Use Case	15
2	3.2.3.2 Actor	15
3	3.2.3.3 Use Case Diagram for User and Admin	16
4	4.2.1 System Architecture Diagram	17
5	4.4.2.1 Class Diagram	22
6	4.5.1.1 Sequence Diagram	24
9	4.5.2.1 Activity Diagram	26
10	4.7.1.1 Component Diagram	30
11	4.7.2.1 Deployment Diagram	31
12	7.1 Landing Page - 1	54
13	7.2 Landing Page - 2	54
14	7.3 Sign Up Page	55
15	7.4 Login Page	55
16	7.5 AgentVerse Dashboard	55
17	7.6 Suggestions - 1	56
18	7.7 Suggestions - 2	56

LIST OF TABLES

S. No.	NAME OF TABLE	PAGE No.
1	3.2.2.1 List of Actors associated and description	13 - 14
2	6.2.3 Test Cases (Unit Testing)	53

CHAPTER 1: INTRODUCTION

Artificial Intelligence has become an important part of modern technology, helping users solve problems across various domains such as education, business, healthcare, and research. Most existing AI systems are designed using a single-model approach, where one system is responsible for handling all types of queries. While this approach works for simple tasks, it often struggles to provide accurate and context-aware solutions for complex and multi-domain problems.

To address this limitation, the concept of multi-agent systems has gained importance. In a multi-agent system, different agents are designed to perform specific tasks based on their roles and expertise. Instead of relying on a single model, multiple agents collaborate to analyze a problem, divide it into smaller tasks, and generate better results. This approach improves the overall efficiency and quality of the system.

With AgentVerse is developed as a multi-agent AI system that can handle a wide range of user queries in an effective manner. The system uses advanced language models to understand user input and generate responses. It is capable of supporting multiple domains such as career guidance, business idea generation, research support, and general problem-solving.

1.1 Purpose of the System

The purpose of the AgentVerse system is to develop an intelligent platform that helps users understand and analyze their ideas, queries, and problems before taking real-world actions. Many users may have plans or decisions to make, but without proper guidance, they may face risks, confusion, or losses. This system helps in reducing such uncertainties by providing structured and well-analysed solutions.

The system aims to provide an effective way for users to solve complex and multi-domain problems by using a multi-agent approach. It analyzes user input, breaks it into smaller tasks, and generates meaningful responses using advanced AI techniques. This helps users gain better clarity and understanding of their queries.

The system also supports better decision-making by offering insights, possible outcomes, and suggestions across different domains such as career and business.

Overall, the purpose of this system is to improve problem understanding, enhance decision-making, and provide reliable guidance, thereby helping users handle real-world challenges more effectively.

1.2 Problem Statement

Most existing AI systems are limited to single-model responses, which often lack depth, specialization, and contextual understanding. When a user submits a complex query—such as business analysis, market strategy, or decision-making—these systems fail to provide comprehensive insights because they do not leverage multiple expert perspectives. Additionally, there is no effective mechanism to reuse past knowledge, leading to repeated computations and inefficient handling of similar queries. This results in slower responses, inconsistent outputs, and limited scalability for real-world applications.

To address these limitations, there is a need for an intelligent multi-agent system that can dynamically select specialized agents, execute them collaboratively, and aggregate their outputs into a structured, high-quality response. The system should also integrate efficient storage mechanisms to manage both structured data and semantic knowledge, enabling faster retrieval and improved decision-making over time. Such a solution must ensure seamless coordination between components, optimize performance, and provide accurate, reusable, and context-aware insights to users.

1.3 Objectives of Study

Here are the Objectives of the Study for AgentVerse:

- To design and develop a **multi-agent AI system** capable of handling complex user queries using specialized agents.
- To implement an **intent detection mechanism** that accurately understands user requirements and
- To dynamically **select relevant agents** based on the identified intent for efficient processing.
- To enable **parallel execution of agents** to generate diverse and expert-level insights.
- To aggregate multiple agent outputs into a **structured and meaningful final response** using CrewAI.

- To integrate **PostgreSQL** for storing structured data such as user requests, agent outputs, and metadata and to utilize **Pinecone** for storing and retrieving vector embeddings for semantic search and knowledge reuse.
- To improve system efficiency by enabling **retrieval of similar past queries** instead of recomputing results.
- To build a scalable backend using **FastAPI** for handling real-time requests and responses.
- To develop a user-friendly frontend interface for seamless interaction with the system.
- To ensure the system provides **accurate, consistent, and context-aware outputs** for better decision-making.

1.4 Scope of the System

The scope of the AgentVerse system includes handling a wide range of user queries across multiple domains such as career guidance, business ideas, research assistance, and general problem-solving. The system is designed to process different types of inputs and provide meaningful, structured, and easy-to-understand responses using a multi-agent approach.

The system can be used by students, professionals, and individuals who need guidance or clarity before making decisions. It helps users analyze their ideas, understand possible outcomes, and gain insights that support better decision-making in real-world situations.

Additionally, it also supports efficient data handling and retrieval to improve response quality and maintain consistency. It can be extended to include more domains, additional agents, and advanced features based on user requirements, making it flexible and scalable.

The system is limited to providing suggestions and guidance based on available data and models, and it does not replace expert advice in critical situations.

Overall, the scope of AgentVerse is to provide an intelligent, adaptable, and user-friendly solution for solving diverse problems effectively.

1.5 Current System

At present, most systems used for solving user queries are based on a single-model or single-agent approach. These systems process user input and generate responses using one AI model, which handles all types of tasks. They are commonly used in chatbots, virtual assistants, and basic AI applications.

Most of these systems focus only on providing direct answers and do not deeply analyze complex or multi-domain problems. Users may receive general responses, but they may not clearly understand the complete solution, possible outcomes, or risks involved in their decisions.

Another limitation is that current systems usually do not break down problems into smaller tasks or involve multiple perspectives. They lack collaboration and cannot simulate how real-world decision-making happens. As a result, users may not get structured guidance or detailed insights needed for better decision-making.

Therefore, the current systems mainly offer basic responses, but they lack advanced features such as multi-agent collaboration, deep analysis, and decision-support capabilities.

1.6 Proposed System

The proposed system, AgentVerse, is designed to overcome the limitations of existing systems by using a multi-agent approach for solving complex and multi-domain user queries. Instead of relying on a single model, the system uses multiple specialized agents that work together to provide more accurate and structured solutions.

The system analyzes user input and divides the problem into smaller tasks, which are assigned to different agents based on their roles. Each agent processes its part of the task and generates relevant outputs. These outputs are then combined to produce a clear and meaningful response for the user.

The system also uses advanced AI techniques such as Large Language Models to understand queries and generate responses. It supports better decision-making by providing insights, suggestions, and possible outcomes, helping users understand their problems more clearly before taking action.

Additionally, the system focuses on improving response quality by efficiently handling and retrieving data. It is designed to be flexible and scalable, allowing it to

support multiple domains and extend its capabilities in the future.

Overall, the proposed system provides a more intelligent, reliable, and user-friendly solution compared to existing systems.

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 Functional Requirements

Functional requirements describe the core operations that the AgentVerse system must perform. These define how the system processes user input, utilizes AI for decision-making, and dynamically creates agents to generate meaningful outputs. The system should perform the following functions

- **User Input Handling**

The system should accept user queries through a REST API and process inputs from different domains such as business ideas, career guidance, and research problems. The system should validate user input and normalize it into a structured format.

- **AI-Based Intent Analysis**

The system should use Large Language Models (LLMs) to analyze the user's query and identify the intent, domain, and required actions.

- **Dynamic Agent Creation**

Based on the LLM's decision, the system should dynamically create specialized agents at runtime using a modular architecture.

- **Task Assignment and Processing**

Each agent should be assigned a specific responsibility such as analysis, planning, or risk evaluation, and process the task independently.

- **Response Generation and Aggregation**

The system should combine outputs from multiple agents and generate a structured and meaningful final response.

- **Error Handling and Fallback Mechanism**

The system should handle failures in AI processing and provide fallback responses to ensure system reliability.

- **API Response and Output Delivery**

The system should return the final output through APIs and present it in a clear and understandable format to the user.

2.2 Non-Functional Requirements

Non-functional requirements define the quality and performance aspects of the AgentVerse system. These requirements ensure that the system operates efficiently, reliably, and securely while providing a good user experience. The system should meet the following requirements:

2.2.1 User Interface and Human Factors

The AgentVerse system should provide a simple and user-friendly interface for interacting with the system.

- **User Interaction:** Users should be able to submit queries through a web interface and receive responses easily.
- **System Feedback:** The system should display responses in a clear and structured format for easy understanding.
- **Ease of Navigation:** Users should be able to access different features of the system without confusion.

2.2.2 Software Requirements

The system requires specific software tools and technologies for development and execution.

- **Operating System:** Windows 10 / Windows 11
- **Technology:** Python, FastAPI, React (Frontend), AI/LLMs
- **Database:** PineCone, PostgreSQL
- **Data Format:** JSON (for API communication)
- **IDE / Workbench:** Visual Studio Code
- **Web Services:** Hugging Face APIs

These components support backend processing, AI integration, and user interaction.

2.2.3 Hardware Requirements

The system requires basic hardware resources to run efficiently.

- **Processor:** Intel i3 or above
- **RAM:** 4 GB (minimum)
- **Hard Disk:** 128 GB
- **Internet Connection:** Required for API calls and AI processing

2.2.4 Usability

Usability ensures that the system is easy to use and accessible for users with different levels of technical knowledge.

- The system should provide a clear and intuitive interface.
- The design should ensure that users can access system features with minimal effort.
- The system should display information in a way that is easy to understand and interpret.

2.2.5 Reliability

Reliability ensures that the system performs consistently and produces accurate results.

- The system should process queries without failure.
- AI responses should be consistent and meaningful.
- Proper error handling mechanisms should be implemented to prevent system failures.

2.2.6 Performance

The system must perform efficiently and quickly.

- The system should process queries within a reasonable time.
- Data retrieval from APIs should be optimized to reduce response time.
- AI processing should minimize delays.

2.2.7 Supportability

Supportability ensures that the system can be maintained and updated easily in the future.

- The system should include proper documentation for developers and users.
- New agents and features should be easily added.
- Technical support should be available to resolve issues and maintain system performance.
- Maintain the System performance

2.2.8 Physical Environment

The system should operate in a normal computing environment with internet connectivity.

- The application should run on standard computers or laptops.
- A stable internet connection is required
- The system should be accessible through web browsers in different environments such as homes, offices, and educational institutions.

2.2.9 Security Requirements

Security is important to protect system data and ensure safe system operation.

- The system should implement authentication and access control mechanisms.
- Data retrieved from web services should be processed securely.
- Security updates and patches should be applied regularly to protect the system from potential threats.

2.2.10 Resource Requirements

The system requires several resources for its operation.

- AI models and APIs for processing queries.
- Computational resources for running backend processes.
- Databases for storing user data and context.

These requirements ensure that the AgentVerse system functions efficiently, securely, and reliably.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Introduction

System Analysis is the process of studying and understanding a system to identify its components, functions, requirements, and possible improvements. It helps in designing or improving a system by analyzing how it operates, what limitations exist in current approaches, and how technology can be used to provide better solutions.

1. Understanding the System

The The AgentVerse system is designed to process and analyze user queries using an AI-based multi-agent approach. The system uses advanced language models to understand user input and dynamically creates specialized agents to handle different tasks. It processes queries in a structured manner to generate meaningful and reliable responses.

The system mainly focuses on analyzing user queries using AI techniques, dynamically creating specialized agents based on the requirements of the query, and breaking down complex problems into smaller tasks through multi-agent processing. It generates structured and meaningful responses to improve user understanding and provides a web-based platform for intelligent problem-solving. This approach enhances the quality of responses and supports better decision-making for users across different domains.

2. System Components

The system consists of the following key components:

- **User Interface (Frontend):**
Developed using React, it provides an interactive and user-friendly interface through which users can submit queries and view responses.
- **Backend Server:**
Implemented using FastAPI, the backend handles user requests, processes input, and manages communication between system components.
- **AI Routing Module (LLM):**
Large Language Models are used to analyze user queries, understand intent, and decide how the query should be processed.

- **External APIs / AI Services:**

The system uses API keys to access external AI services. API is used for accessing Large Language Models to perform query understanding, intent analysis, and response generation. These APIs enable intelligent processing and routing of user queries within the system.

- **Database:**

Pinecone is used as a vector database for storing and retrieving contextual information, and PostgreSQL is used for storing structured data such as user details and interaction history.

- **Data Format:**

JSON format is used for communication between frontend, backend, and external APIs.

3. System Functions

The system performs several key operations for processing and analyzing user queries:

- **User Query Processing:**

The system receives and validates user input through APIs for further processing.

- **Intent Analysis:**

The system analyzes the query using Large Language Models to understand its purpose and domain.

- **Agent Creation:**

The system dynamically creates specialized agents based on the requirements of the query.

- **Task Processing:**

Each agent processes its assigned task such as analysis, planning, or evaluation.

- **Response Generation:**

The system combines the outputs from all agents to generate a structured and meaningful response.

4. Identified Problems in Existing Systems

- Many traditional AI systems rely on a single-model approach and do not handle complex or multi-domain queries effectively.

- Existing systems often provide direct answers without performing deep analysis or breaking down the problem into smaller tasks.
- Most platforms do not support multi-agent collaboration, which limits their ability to generate structured and well-analyzed solutions.
- Users may not receive clear insights, possible outcomes, or risk analysis, making it difficult to make informed decisions.

5. Proposed Solutions & System Improvements

- Use of Large Language Models (LLMs) to analyze user queries and understand intent more effectively.
- Implementation of a multi-agent approach where specialized agents collaborate to solve complex and multi-domain problems.
- Dynamic creation of agents at runtime based on the requirements of the user query.
- Development of a web-based platform that allows users to interact with the system and receive structured responses easily.
- Integration of backend APIs to process requests, manage data, and ensure smooth communication between system components.
- The system workflow involves receiving user input, analyzing the query using AI, creating and assigning tasks to agents, processing each task, and combining the results to generate a final structured response for the user.

3.2 Use Cases

Use cases describe the interactions between users and the AgentVerse system. They explain how users interact with the system to submit queries and receive structured responses. In the AgentVerse system, use cases illustrate how users engage with the platform to analyze their ideas, solve problems, and obtain insights across different domains.

The use case model helps in identifying system functionalities and the responsibilities of each component involved in processing user queries.

3.2.1 Actors

Actors represent the entities that interact with the AgentVerse system. The system mainly involves two actors:

1. User

The user is the primary actor who interacts with the system to analyze queries and obtain meaningful solutions. The user can perform the following actions:

- Register and login to the system
- Submit queries or problems related to different domains
- Receive structured responses and insights
- View suggestions, analysis, and possible outcomes
- Interact with the system through the web interface

2. Admin

The admin manages and maintains the system. The admin is responsible for monitoring system performance and ensuring smooth operation. The admin can perform the following actions:

- Manage user data and system configurations
- Update and maintain AI models and agents
- Ensure proper functioning of backend services and databases.

3.2.2 List of Use Cases

The following table shows the main use cases of the AgentVerse system.

Table 3.2.2.1: List of Actors associated and description

Use Case ID	Usecase Name	Actor	Description
UC1	Register / Login	User	Allows users to create an account and log in to access system features.
UC2	Submit Query	User	User enters a query or problem.
UC3	View Response	User	Displays structured responses, insights, and suggestions generated by the system.

UC4	Analyze Query	System	The system processes and understands the user query using AI techniques.
UC5	Identify Intent	System	Determines the purpose and domain of the query using LLMs.
UC6	Create Agents	System	Dynamically creates specialized agents based on query requirements.
UC7	Process Tasks	System	Assigns tasks to agents and processes them independently.
UC8	Generate Response	System	Combines outputs from multiple agents to produce a final response.
UC9	Store Data	System	Stores user data and interaction history in databases.
UC10	Retrieve Context	System	Retrieves relevant information to improve response quality.

3.2.3 Use Case Diagram

Use case diagram is created to visualize the relationships between actors and use cases. A use case is a pattern of behaviour the system exhibits. Each use case is a sequence of related transactions performed by an actor and the system. Diagrammatically actor and use case are represented by stick figure and oval respectively.

- **Symbols**



Fig 3.2.3.1: New Use Case

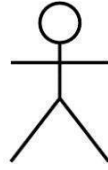


Fig 3.2.3.2: Actor

- **Use cases:** Horizontally shaped ovals that represent the different uses that a user might have.
- **Actors:** Stick figures that represent the people actually employing the use cases.
- **Associations:** A line between actors and use cases. In complex diagrams, it is important to know which actors are associated with which use cases.
- **System boundary boxes:** A box that sets a system scope to use cases. All use cases outside the box would be considered outside the scope of that system. For example, Psycho Killer is outside the scope of occupations in the chainsaw example found below.
- **Packages:** A UML shape that allows you to put different elements into groups. Just as with component diagrams, these groupings are represented as file folders.
- **Actors:** The users that interact with a system. An actor can be a person, an organization, or an outside system that interacts with your application or system. They must be external objects that produce or consume data.
- **System:** A specific sequence of actions and interactions between actors and the system. A system may also be referred to as a scenario.
- **Goals:** The end result of most use cases. A successful diagram should describe the activities and variants used to reach the goal.

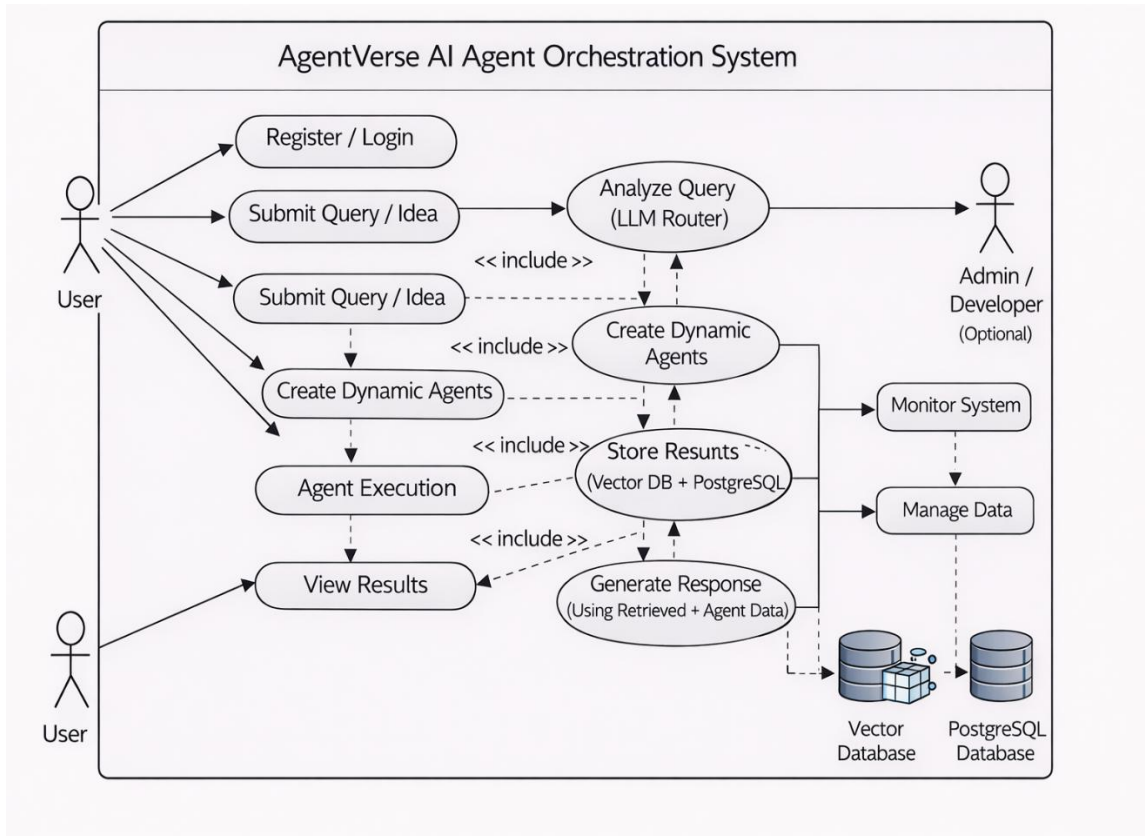


Fig 3.2.3.3: Use Case Diagram for User and Admin

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

System design describes how the proposed system is structured and how its components interact with each other to achieve the required functionality. It provides a blueprint for implementing the system by defining system architecture, data flow, modules, and interactions between components.

The AgentVerse AI Agent Orchestration System is designed to receive user queries through a web interface, process them using backend services, and analyze them using Large Language Models. Based on the query, the system dynamically creates specialized agents that perform tasks such as analysis, planning, and evaluation. The system also utilizes databases to store and retrieve contextual information, improving response quality and efficiency.

The system design includes various models such as system architecture, object models, dynamic models, database design, and static models. These models help in understanding the structure and behaviour of the system.

4.2 System Architecture

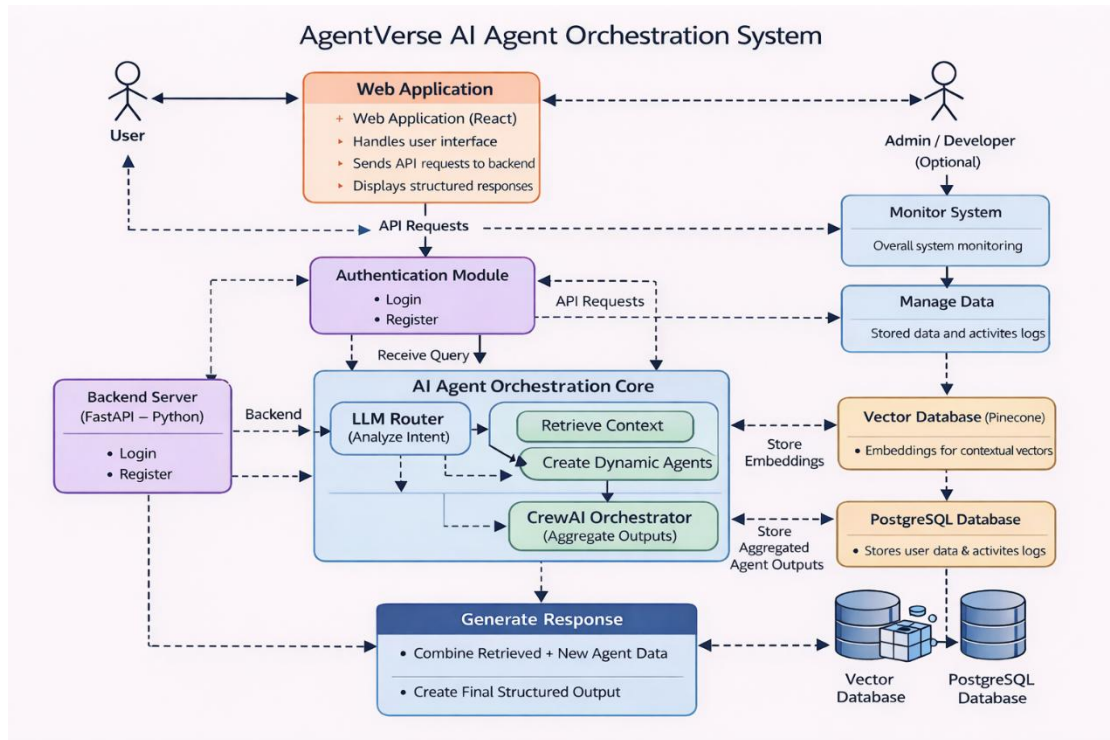


Fig 4.2.1: System Architecture Diagram

4.3 System Object Model

4.3.1 Introduction

The System Object Model describes the structure of the system in terms of objects, classes, and their relationships. It explains how different components of the AgentVerse system interact with each other to perform various functions.

In AgentVerse, the object model represents the major components such as the user interface, backend server, agent manager, AI agents, task orchestrator, and memory module. These components work together to receive user queries, process them using multiple AI agents, coordinate tasks, and generate intelligent responses.

The object model helps in understanding how information flows within the system and how different modules interact and communicate with each other.

4.3.2 Subsystems

The AgentVerse is divided into several subsystems that perform specific tasks.

1. User Interface Subsystem

- Provides interaction between the user and the system.
- Allows users to input queries and view generated responses.
- Displays outputs from multiple AI agents in a structured format.

2. Authentication Subsystem

- Manages user login and session management.
- Ensures secure access to the application.
- Maintains user-specific interaction history.

3. API Gateway Subsystem

- Acts as a bridge between frontend and backend services.
- Handles request routing and response delivery.
- Ensures smooth communication between system components.

4. Agent Management Subsystem

- Responsible for creating and managing multiple AI agents.

- Controls agent lifecycle and coordination.

5. Task Orchestration Subsystem

- Assigns tasks to appropriate AI agents.
- Manages execution flow and combines results.

6. AI Agent Processing Subsystem

- Executes tasks using specialized AI agents.
- Enables collaboration between multiple agents.

7. LLM Integration Subsystem

- Connects the system to Large Language Models.
- Sends prompts and retrieves generated responses.
- Supports different models for flexible AI processing.

8. Database Subsystem

- Stores user data, session details, and system logs.
- Used for managing user-related information securely.
- Supports efficient data retrieval and storage.

4.4 Object Description

4.4.1 Objects

The major objects used in the system are described below.

1. User Object

- Represents the system user.
- Stores user details such as user ID, username, and session data.
- Allows users to interact with the system by submitting queries.

2. AuthService Object

- Handles user authentication and authorization.
- Validates login credentials.
- Manages secure access.

3. Session Object

- Maintains active user sessions.
- Stores session-specific data such as interaction history.

4. WebApp Object

- Acts as the main controller of the application.
- Handles user input and sends API requests to the backend.

5. BackendService Object

- Processes user requests and manages system workflow.
- Coordinates communication between AI modules and databases.

6. LLMRouter Object

- Analyzes user queries to understand intent and context.
- Routes queries to appropriate processing components.

7. AIAgent Object

- Represents dynamically created agents for specific tasks.
- Performs operations such as analysis, planning, or evaluation.

8. Orchestrator Object

- Manages multiple agents and controls task execution flow.
- Combines outputs from agents for final processing.
- Represents dynamically created agents for specific tasks.
- Performs operations such as analysis, planning, or evaluation.

9. VectorDB Object (Pinecone)

- Stores embeddings and contextual data.
- Retrieves relevant information for improving responses.

10. PostgreSQL DB Object

- Stores structured data such as user details and query history.
- Maintains logs and system records.

11. ResponseGenerator Object

- Combines retrieved data and agent outputs.
- Generates the final structured response for the user.

4.4.2 Class Diagrams

The class diagram represents the static structure of the system. It shows the classes used in the application, their attributes, methods, and the relationships between them. These classes work together to process user queries, manage system operations, and generate structured responses.

Important classes include:

- User
- AuthService
- WebApp
- BackendService
- LLMRouter
- AgentOrchestrator
- Agent
- VectorDB
- PostgreSQLDB
- ResponseGenerator

The relationships between these classes illustrate how different modules interact within the system. For example, the WebApp class communicates with the BackendService to send user requests. The BackendService interacts with the LLMRouter to analyze user queries and determine intent. Based on this, the AgentOrchestrator dynamically creates and manages multiple Agent objects. These agents process tasks, and their outputs are stored and retrieved using the VectorDB and PostgreSQLDB.

The ResponseGenerator class combines retrieved context and agent outputs to generate the final structured response, which is then sent back to the user through the WebApp.

The class diagram provides a clear view of system architecture, object relationships, and how different components collaborate to deliver intelligent results.

The class diagram provides a clear view of system architecture and object relationships.

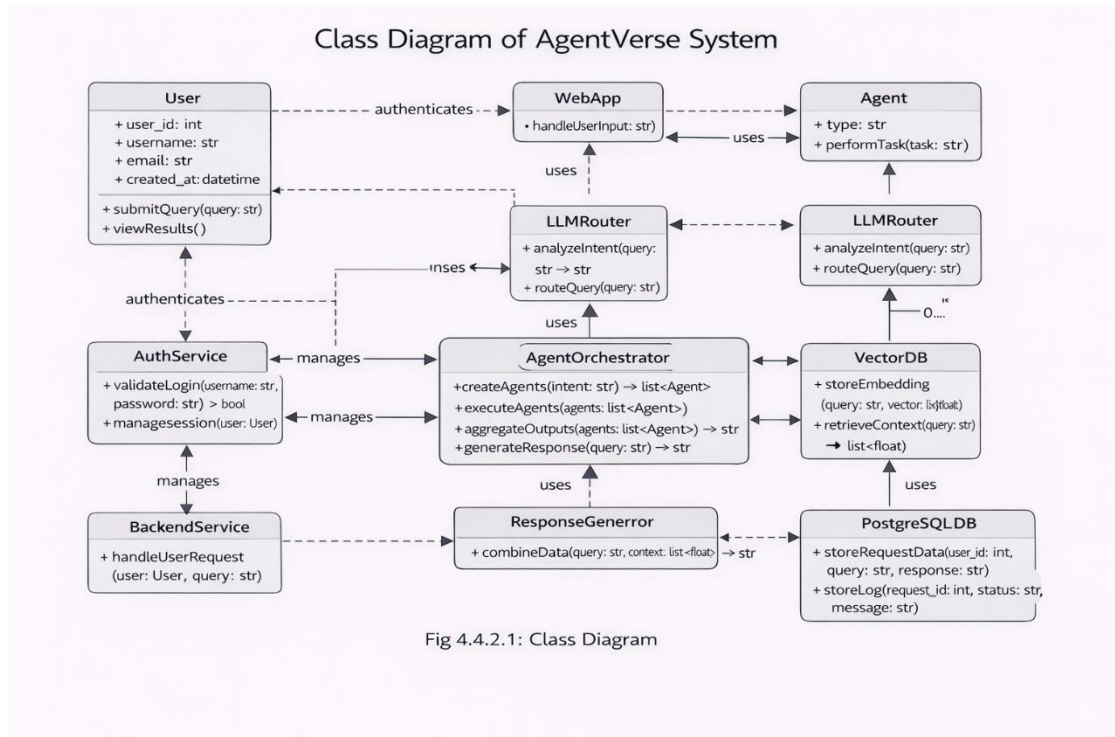


Fig 4.4.2.1: Class Diagram

4.5 Dynamic Model

The dynamic model represents the behaviour of the system during execution. It shows how the system responds to user actions and processes requests.

4.5.1 Sequence Diagram

A sequence diagram is a type of interaction diagram because it describes how and in what order a group of objects works together. A sequence diagram shows object interactions arranged in time sequence. The interaction that takes place in collaboration that either realizes a use case or an operation. Sequence diagrams provide high level interactions between user of the system and the system, between the system and the other systems, or between subsystems.

The following scenarios describe how the system operates using sequence diagrams.

These scenarios help in understanding the interaction between different components of the AgentVerse platform.

- **Usage scenario:** A usage scenario shows how the system is used by the user. In this system, the user submits a query through the interface. The system analyzes the query, creates dynamic AI agents if required, and generates a structured response after processing.
- **Method logic:** This explains the internal working of the system. The backend server receives the query, sends it to the LLM for intent analysis, and determines the required agents. The agents process tasks independently, and their outputs are aggregated to produce the final response.
- **Service logic:** The system provides services such as authentication, agent orchestration, and data storage. The authentication module manages login and access control, while the databases store user data, embeddings, and system logs. The orchestration module ensures coordination between agents.
- **Data processing logic:** The user query is processed by the LLM to extract intent and metadata. Relevant context is retrieved from the vector database if available. AI agents process subtasks, and their outputs are stored and combined to generate meaningful results.
- **System interaction:** The web application interacts with the backend server, LLM module, agent manager, CrewAI orchestrator, and databases. All components work together to process queries, coordinate agents, and deliver structured responses to the user.

The sequence diagram illustrates the step-by-step interaction between system components during query processing and response generation.

The process includes:

1. User submits a query through the web application.
2. The Web Application sends the request to the FastAPI backend.

3. The backend validates input and generates metadata.
4. The query is sent to the LLM for intent analysis.
5. The system checks the vector database for existing context.
6. If required, dynamic AI agents are created by the Agent Manager.
7. The Task Orchestrator assigns subtasks to different agents.
8. AI agents process tasks independently using LLM support.
9. CrewAI coordinates and aggregates outputs from all agents.
10. Results are stored in the vector database and relational database.
11. The final structured response is sent back to the user interface.

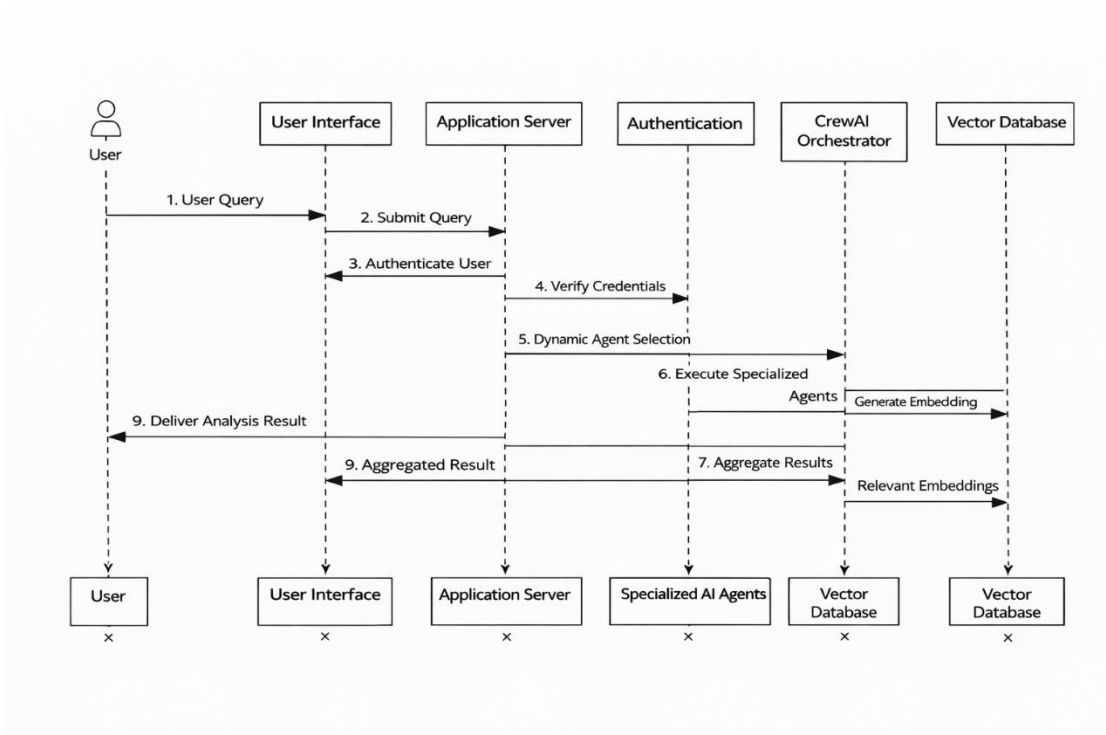


Fig 4.5.1.1: Sequence Diagram

4.5.2 Activity Diagrams

An activity diagram is another important diagram in UML to describe the dynamic aspects of the system. An activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system. The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of

flow control by using different elements such as fork, join, etc. The basic purposes of activity diagrams are similar to the other four diagrams. It captures the dynamic behavior of the system. Other four diagrams are used to show the message flow from one object to another but the activity diagram is used to show message flow from one activity to another.

The activity diagram represents the workflow of the AgentVerse platform. The main activities include:

1. User opens the web application.
2. User submits a query through the interface.
3. The system validates the input and generates metadata.
4. The query is sent to the LLM for intent analysis.
5. The system checks the vector database for existing context.
6. If relevant context is available, it is retrieved and used.
7. If required, dynamic AI agents are created based on query intent.
8. The task is divided into subtasks by the orchestrator.
9. AI agents process tasks independently using LLM support.
10. Outputs from all agents are aggregated using CrewAI.
11. The system stores embeddings in the vector database and logs in the relational database.
12. The response generator combines agent outputs and context.
13. The final structured response is generated.
14. The system displays the response to the user.

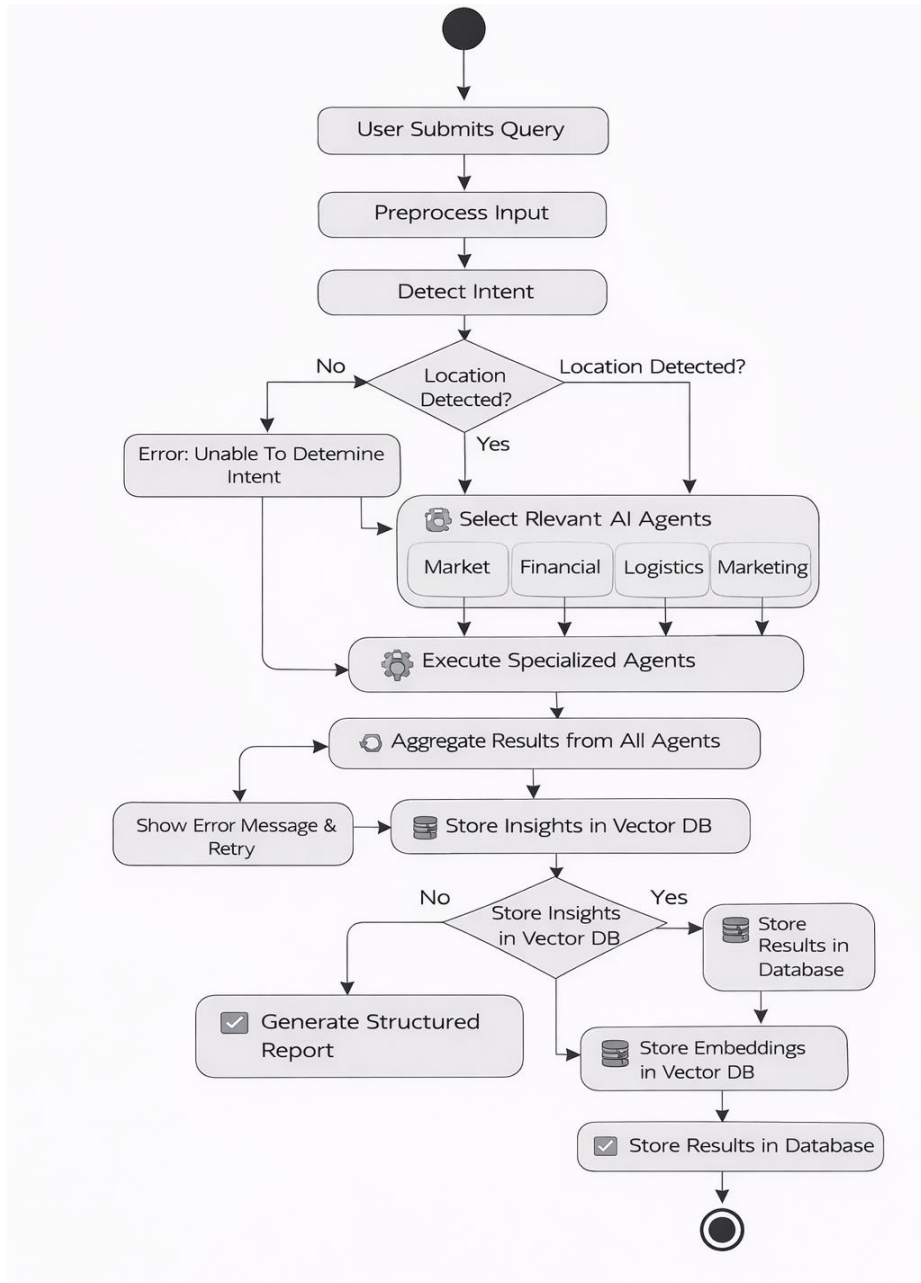


Fig 4.5.2.1: Activity Diagram

4.6 Database Design

Database design defines how data is organized, stored, and managed within the system. The AgentVerse platform follows a hybrid database architecture to ensure efficient storage, fast retrieval, and context-aware processing.

The system uses PostgreSQL as a relational database to store structured data. PostgreSQL is robust, scalable, and well-suited for handling user information, request metadata, and system logs.

The relational database is primarily used for storing user-related data such as user identification details, authentication credentials, query logs, and request metadata. This data supports authentication, session management, tracking user interactions, and maintaining system records.

In addition to the relational database, the system uses Pinecone as a vector database. Pinecone is used to store embeddings generated from user queries and AI agent outputs. These embeddings enable efficient similarity search and retrieval of relevant past context, which improves the quality and consistency of responses.

The system also utilizes JSON (JavaScript Object Notation) as a data exchange format. User queries, LLM responses, and agent outputs are processed in JSON format during runtime for seamless communication between different system components.

The AgentVerse system does not rely solely on stored data for generating responses. Instead, it dynamically processes user queries using Large Language Models and retrieves relevant context from the vector database when required. This ensures that responses are both context-aware and up-to-date.

This design reduces storage overhead, improves retrieval efficiency, and enhances the overall performance of the system by combining structured data storage with intelligent context retrieval.

4.6.1 Database Tables

1. User Table

- user_id (Primary Key)
- contact (email/mobile – unique)
- name

2. Query Log Table

- query_id (Primary Key)
- user_id (Foreign Key)
- query_text
- intent
- timestamp

3. Agent Output Table

- output_id (Primary Key)
- query_id (Foreign Key)
- agent_role
- output_text

4. Metadata Table

- metadata_id (Primary Key)
- query_id (Foreign Key)
- intent
- processing_time

5. Vector Database (Pinecone) Storage

Stores embeddings of:

- user queries
- agent outputs
- contextual data

Used for similarity search and context retrieval

6. JSON Usage

Used as a data exchange format between frontend, backend, and AI modules

Stores:

- query data
- agent responses
- metadata

4.7 Static Model

The static model represents the structural components of the system and how they are connected. It describes the organization of different modules without focusing on the dynamic behaviour of the system. The static model helps in understanding how the system is designed and how different components interact with each other.

In the AgentVerse system, the static model includes components such as the user interface, backend server, LLM module, agent manager, task orchestrator, vector database (Pinecone), and relational database (PostgreSQL). The backend server acts as the central component that connects all modules and processes user requests.

The model also includes data storage components where PostgreSQL is used for storing user data, query logs, and metadata, while Pinecone is used for storing embeddings and contextual information. Overall, the static model helps in understanding how different components are organized and work together to enable intelligent multi-agent query processing and decision support.

4.7.1 Component Diagrams

The component diagram represents the major modules of the multi-agent AI system and their interactions.

Main components include:

- User Interface
- Authentication Module (Login/Register)
- Application Server (FastAPI Backend)
- CrewAI Orchestration Module
- Specialized AI Agents
- Aggregation Module (Final Report Generator)
- Vector Database (Pinecone)
- Relational Database (PostgreSQL)
- Language Model APIs

These components work together to accept user queries, dynamically select and execute relevant AI agents, aggregate their outputs into a structured report, and store both structured and semantic data for future retrieval and intelligent recommendations.

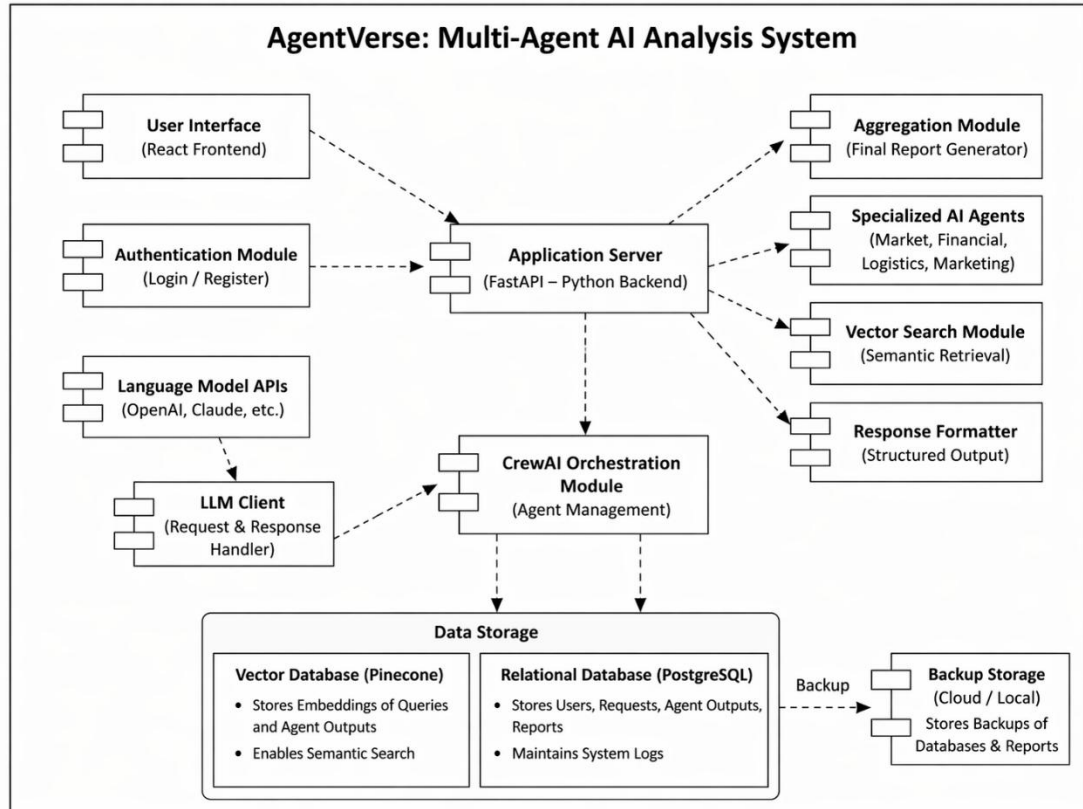


Fig 4.7.1.1: Component Diagram

4.7.2 Deployment Diagrams

The deployment diagram represents the physical architecture of the system. The system includes the following nodes:

User Device

- Web browser used by the user to access the system.
- Sends user queries and displays analysis results.

Admin Device

- Used for system monitoring and model updates.

Web Server

- Handles incoming HTTP requests from the frontend.
- Manages agent execution, aggregation.
- The web server also manages communication between components which is very crucial for agents.

CrewAI Engine

- Responsible for orchestrating multiple AI agents.
- Dynamically selects and executes agents based on user query intent.

Language Model API Service

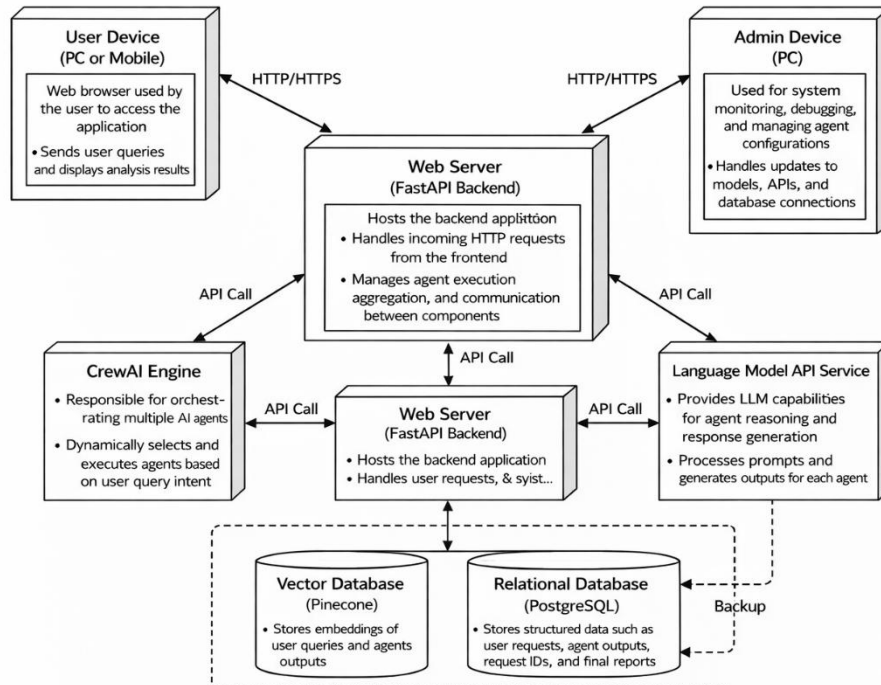
- Processes prompt and generate outputs for each agent.
- Provides LLM capabilities for agent reasoning and response generation.

Vector Database (Pinecone)

- Stores embeddings of user queries and agent outputs
- Enables semantic search and retrieval of similar past queries.

Relational Database (PostgreSQL)

- Stores structured data such as user requests, agent outputs, request IDs, and final reports.
- Maintains history and supports data management operations.



• Fig 4.7.2.1: Deployment Diagram

CHAPTER 5: IMPLEMENTATION

5.1 Selected Software

1. Programming Language

❖ Python

Python is a high-level, interpreted, interactive and object-oriented scripting language Python is designed to be highly readable. It uses English words frequently whereas other languages use punctuation, and it has fewer syntactic constructions than other languages.

Python Features:

- Easy-to-learn-Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- Easy-to-read-Python code is more clearly defined and visible to the eyes.
- Easy-to-maintain-Python's source code is fairly easy-to-maintain
- A broad standard library - Python's bulk of the library is very portable and cross platform compatible on UNIX, Windows, and Macintosh.
- Interactive Mode - Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- Portable - Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- Extendable- You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- Databases-Python provides interfaces to all major commercial databases
- GUI Programming -Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems.

Advantages of Python:

- Easy to Learn & Use – Simple syntax, making it beginner-friendly.
- Readable & Maintainable Code – Clean structure enhances code readability.

- Extensive Libraries & Frameworks – Supports libraries like NumPy, Pandas, TensorFlow, Django, Flask, etc.

Disadvantages of Python:

- High Memory Usage – Not suitable for memory-intensive tasks due to dynamic typing and garbage collection.
- Weak in Mobile Development – Less commonly used for mobile applications compared to Java or Swift.

Install Python Step-by-Step in Windows:

Step 1: Download Python

- Visit the official Python website: <https://www.python.org/downloads/>
- Click on the Download Python (latest version) button.
- Ensure you download the Windows installer (64-bit or 32-bit based on your system).

Step 2: Run the Installer

- Locate the downloaded .exe file and double-click to open it.

Step 3: Customize Installation (Optional but Recommended)

- Check the box "Add Python to PATH" (Important for running Python from the command line).
- Click on "Customize Installation" if you want to select additional features. Otherwise, click "Install Now".

Step 4: Install Python

- Click Install and wait for the process to complete.

Step 5: Verify Installation

- Open Command Prompt (cmd) or Windows PowerShell.
- Type `python --version` and press Enter.
- If installed correctly, it will display the Python version.

2. Frameworks

❖ FastAPI

FastAPI is a modern, high-performance web framework written in Python. It is used to build APIs quickly and efficiently with automatic validation and interactive documentation. FastAPI is designed for speed and supports asynchronous programming, making it suitable for scalable and real-time applications.

In this project, FastAPI is used to develop the backend server that processes user queries, performs intent analysis using LLMs, manages multi-agent workflows, interacts with databases, and returns structured responses to the frontend interface.

Features of FastAPI

- High performance comparable to Node.js and Go
- Built-in data validation using Pydantic
- Automatic API documentation (Swagger UI)
- Supports asynchronous request handling
- Easy integration with AI/ML models and databases

Advantages

- Fast and efficient for API development
- Supports async operations for better performance.
- Automatic documentation reduces development effort.

Disadvantages

- Smaller community compared to older frameworks like Django
- Requires understanding of async programming.

3. Multi-Agent Framework

❖ CrewAI

CrewAI is a multi-agent orchestration framework used to design and manage collaborative AI agents. It enables multiple specialized agents to work together by assigning roles, goals, and tasks, allowing complex problems to be solved through coordinated execution.

In this project, CrewAI is used to create and manage dynamic AI agents that perform tasks such as analysis, planning, and evaluation. It coordinates the interaction between agents, aggregates their outputs, and ensures that the final response is structured and meaningful.

Features

- Supports multi-agent collaboration and coordination.
- Role-based agent design (analysis, planning, evaluation, etc.).
- Task delegation and workflow management.
- Enables structured and modular AI workflows

Advantages

- Improves problem-solving through agent collaboration.
- Supports scalable and modular system design.
- Flexible and customizable agent workflows.

Disadvantages

- Requires proper design of agent roles and tasks.
- Performance depends on LLM response quality.

4. Data Processing Libraries

❖ Pandas

Pandas is a powerful Python library used for data manipulation and analysis. It provides data structures such as DataFrames and Series that simplify data handling.

In this project, Pandas is used for loading the dataset, cleaning data, handling missing values, and preparing features for the machine learning model.

Features

- DataFrame-based data structure.
- Easy data cleaning and preprocessing.
- Supports multiple file formats such as CSV and Excel.

❖ NumPy

NumPy is a numerical computing library in Python used for performing mathematical and scientific calculations. It provides support for large multi-dimensional arrays and matrices.

In this project, NumPy is used for performing numerical computations and data transformations during the machine learning process.

Features

- High-performance numerical operations.
- Multi-dimensional array support.
- Integration with machine learning libraries.

5. Visualization and Interface

❖ React

React and Vite are used to develop the frontend user interface of the system. These technologies help in creating a fast, interactive, and user-friendly interface where users can submit queries and view structured AI-generated responses.

React is used to build reusable UI components and manage the application state efficiently. It enables dynamic rendering of content based on user interactions. Vite is used as the build tool, providing fast development and optimized performance with features like hot module replacement and quick server startup.

Features

- Component-based architecture for reusable UI elements
- Fast rendering using Virtual DOM
- State management for dynamic data handling
- Vite provides fast build and development environment
- Supports modern JavaScript features.

6. LLMs Used

❖ LLaMA 3.1 (8B Instruct)

LLaMA 3.1 (8B Instruct) is a Large Language Model provided through Hugging Face APIs. It is designed for instruction-based tasks such as understanding user queries, generating responses, and assisting in decision-making processes.

The model is optimized to follow prompts effectively and produce structured and meaningful outputs.

In this project, the LLaMA 3.1 (8B Instruct) model is used for intent analysis, query understanding, and generating responses for individual AI agents. It processes user input, supports dynamic agent workflows, and helps in producing accurate and context-aware outputs.

Features

- Instruction-tuned model for better prompt understanding.
- Supports natural language processing and text generation.
- Capable of reasoning, summarization, and analysis.

7. Development Tools

❖ Visual Studio IDE

Visual Studio Code (VS Code) is a free, lightweight, and open-source code editor developed by Microsoft. It supports multiple operating systems, including Windows, macOS, and Linux.

VS Code is designed to be versatile and user-friendly, offering features such as:

- Syntax Highlighting and Autocompletion: Enhances code readability and writing efficiency.
- Integrated Debugging: Allows developers to test and debug code within the editor.
- Built-in Git Integration: Facilitates version control and collaboration directly from the editor.
- Extensibility: Supports a wide range of extensions to add functionalities like new language support, themes, and tools.

8. Database

❖ Pinecone (Vector Database)

Pinecone is a vector database used to store and retrieve high-dimensional embeddings efficiently. It is designed for similarity search, enabling the system to find relevant information based on semantic meaning rather than exact keyword matching.

In this project, Pinecone is used to store embeddings of user queries and AI agent

outputs. When a new query is received, the system searches the vector database to retrieve relevant past context, which helps in generating more accurate and context-aware responses.

Features:

- High-performance vector similarity search
- Scalable and managed cloud-based database
- Efficient handling of high-dimensional embeddings
- Easy integration with AI and machine learning systems

Advantages:

- Enables semantic search instead of keyword-based search
- Improves response quality using contextual memory
- Fast and efficient retrieval of relevant data

❖ **Pinecone (Vector Database)**

PostgreSQL is a powerful, open-source relational database management system used for storing structured data. It supports advanced querying, data integrity, and scalability, making it suitable for modern web applications.

In this project, PostgreSQL is used to store user data, authentication details, query logs, and system metadata. It helps in managing user sessions, tracking interactions, and maintaining structured records required for system operations.

Features:

- Supports structured data with relational tables
- Advanced querying using SQL
- High reliability and data integrity
- Scalable for large applications

Advantages:

- Efficient handling of structured data
- Ensures data consistency and security

❖ JSON (JavaScript Object Notation)

JSON is used as a backup storage mechanism in this system. It stores user profile data in a structured key-value format.

In this project, JSON is used as a data exchange format to enable communication between different system components. It is used to send and receive data between the frontend (React) and backend (FastAPI), ensuring smooth interaction. JSON is also used to store user queries, API requests, and responses in a structured format. Additionally, it represents AI agent outputs and metadata, and facilitates communication between the LLM, agents, and backend modules.

Advantages:

- Easy to read and write
- Lightweight format
- No database dependency

5.2 Source Code

The AgentVerse Multi-Agent AI Analysis System is implemented using Python and the FastAPI framework. The system integrates large language models, multi-agent orchestration using CrewAI, and vector-based memory storage using Pinecone to analyze user queries and generate structured, intelligent reports.

The source code is organized into multiple modules responsible for input processing, request handling, intent detection, dynamic agent execution, and result aggregation. Specialized agents such as market, financial, logistics, and marketing agents independently process the user query, and their outputs are combined using an aggregation agent to produce a final structured response. The system also utilizes LiteLLM to connect with different language models.

The backend communicates with the frontend interface to receive user queries and display the generated insights. Additionally, the system stores structured outputs in a relational database and uses vector embeddings for semantic search and knowledge retrieval, enabling smarter and more context-aware responses over time.

The implementation consists of the following major modules:

1. Backend Server Initialization

The backend server is implemented using the FastAPI framework. It handles incoming HTTP requests, routes them through different processing steps, and returns structured AI-generated responses to the frontend.

The backend initializes the application and connects all pipeline steps.

Sample source code

```
from flask import Flask, request, jsonify, render_template

from fastapi import FastAPI

from routers. analyze import router as analyze_router

app = FastAPI ()

app.include_router(analyze_router)

app = Flask(__name__)

with open ("ai_aqi_model_spatial.pkl", "rb") as f:
    model = pickle. load (f)
```

This code initializes the FastAPI application and connects the API routes responsible for handling user queries.

2. Input Processing Module (Step-1)

This module processes and cleans the user input before passing it into the system pipeline.

Sample Source code

```
def process_input(user_message):

    return {

        "received": True,
```



```

        "user_message": user_message.strip()

    }

```

This ensures consistent and clean input for further processing.

3. Request Handling Module (Step-2)

This module generates a unique request ID and structures the request for tracking.

Sample Source code

```

import uuid
from datetime import datetime

def handle_request(message):
    return {
        "request_id": str(uuid.uuid4()),
        "cleaned_message": message,
        "status": "accepted",
        "timestamp": datetime.utcnow().isoformat()
    }

```

This enables traceability across the system

4. Dynamic Agent Execution Module

This module executes multiple specialized AI agents asynchronously.

Sample Source Code

```

from agents.agent_factory import AgentFactory

import asyncio

async def create_dynamic_agents(step3_output: dict, input_message: str,
context=None) -> dict:

```

```

request_id = step3_output["request_id"]

intent = step3_output["intent"]

agent_names = step3_output["agents"]

agents = AgentFactory.create_agents(

    agent_names=agent_names,

    intent=intent,

    request_id=request_id

)

execution_results = []

# Prepare memory context text

memory_text = ""

if context:

    for m in context:

        query = m.get("query")

        answer = m.get("answer")

        if query and answer:

            memory_text += f"Previous Query: {query}\n"

            memory_text += f"Previous Answer: {answer}\n\n"

for agent in agents.values():

    agent_input = {

```

```

        "input": input_message
    }

    if memory_text:

        agent_input["memory_context"] = memory_text

    result = await agent.execute(agent_input)

    execution_results.append(result)

return {

    "request_id": request_id,

    "intent": intent,

    "agents_executed": agent_names,

    "results": execution_results,

    "status": "completed"

}

```

Each agent generates its own analysis independently.

5. Aggregation Module

The aggregation module uses CrewAI to combine outputs from all agents into a structured report.

Sample Source Code

```

from crewai import Task

from .crew_agents import aggregator_agent

def create_aggregation_task(agent_results):

```

```

combined = "\n\n".join(

    f'{r['agent_name']}: \n {r['analysis']}'

    for r in agent_results

)

task = Task(

    description=f"""

```

You are an AI system that organizes outputs from multiple agents.

Do NOT rewrite everything into a report.

Instead, present the results grouped by agent.

Format strictly like this:

Agent Name:

- key point

- key point

- key point

Agent Name:

- key point

- key point

Final Conclusion:

2 concise sentences summarizing the overall insights.

Agent Outputs:

```

{combined}

"""

    agent=aggregator_agent,

    expected_output="Agent-wise structured summary"

)

return task

```

The aggregator produces:

- Executive Summary
- Key Insights
- Actionable Recommendations

This is the final output shown to the user.

6. Database Storage Module (PostgreSQL)

The system stores structured results in a relational database for persistence and tracking.

Sample Source Code

```

import psycopg2

def store_request(data):

    conn = psycopg2.connect(

        host="localhost",

        database="agentverse_db",

        user="postgres",

        password="yourpassword",

```

```

        port=5432

    )

    cur = conn.cursor()

    query = """

    INSERT INTO requests

    (request_id, user_query, intent, final_output)

    VALUES (%s,%s,%s,%s)

    """

    cur.execute(query, (

        data["request_id"],

        data["query"],

        data["intent"],

        data["output"]

    ))

    conn.commit()

    cur.close()

    conn.close()

```

This enables:

- Similar query detection
- Knowledge reuse
- Context-aware responses

7. Vector Memory Module (Pinecone)

The system uses Pinecone to store embeddings for semantic search and memory retrieval.

Sample Source Code

```
from database.postgres_client import store_request

from database.pinecone_client import store_embedding

from database.pinecone_client import store_embedding, search_memory

def run_step5(step3, agent_results, query):

    memory = search_memory(query)

    # if exact same query already exists

    if memory and memory[0]["score"] > 0.9:

        return memory[0]["answer"]

    final_output = aggregate_agent_outputs(agent_results)

    data = {

        "request_id": step3["request_id"],

        "query": query,

        "intent": step3["intent"],

        "output": final_output

    }

    store_request(data)

    store_embedding(step3["request_id"],

                    query,
```

```
        final_output)

    return final_output
```

This enables:

- Similar query detection
- Knowledge reuse
- Context-aware responses

8. Frontend Interaction Module

The frontend communicates with the backend using API calls.

Sample Source Code

```
async function analyzeQuery() {

    const response = await fetch("/analyze", {

        method: "POST",

        headers: {"Content-Type": "application/json"},

        body: JSON.stringify({query: userInput})

    });

    const data = await response.json();

    console.log(data);

}
```

This sends user queries and displays the structured AI response.

❖ Project File Structure

The implementation includes several important source code files:

- **main.py** – Entry point of the FastAPI backend. Handles API routes like /analyze and controls the full pipeline execution (Step 1 → Step 5).
- **step1_intent.py** – Identifies user intent from the input query. Decides what the user is asking (business idea, analysis, strategy, etc.).
- **step2_agent_selector.py** – Selects relevant agents dynamically based on the identified intent.
- **step3_agent_execution.py** – Executes selected agents asynchronously and collects their outputs.
- **step4_output_formatter.py** – Structures agent outputs into a consistent JSON format for further processing.
- **step5_aggregator/** – Handles final aggregation using CrewAI:
 - **aggregator.py** – Runs CrewAI crew and manages aggregation flow
 - **crew_agents.py** – Defines aggregator agent (LLM configuration)
 - **crew_tasks.py** – Creates aggregation task combining all agent outputs
- **database/** – Handles storage and retrieval:
 - **postgres_client.py** – Stores structured data (request_id, intent, outputs)
 - **pinecone_client.py** – Stores and retrieves vector embeddings for semantic search
 - **rub_step5.py** – Connects aggregation output with database storage
- **models/** – Contains schemas and data models for request/response handling.
- **frontend/** – React-based UI that interacts with backend APIs and displays results.
- **utils/** – Helper functions (ID generation, formatting, validation, etc.).
- **.env** – Stores API keys (OpenAI, Pinecone, etc.) and environment variables.

CHAPTER 6: TESTING

6.1 Introduction

Testing is an important phase in the software development process. It ensures that the developed system functions correctly and meets the specified requirements. Software testing helps identify errors, bugs, and performance issues before the system is deployed.

In the AgentVerse project, testing is conducted to ensure that all agents operate correctly within the orchestration pipeline, that the FastAPI backend handles requests reliably, and that the React frontend properly displays structured responses. Special attention is given to validating inter-agent communication, response formatting consistency, and error handling when external APIs or LLM services fail or return unexpected outputs.

The testing process also verifies that the system maintains stability under multiple user queries, correctly routes tasks to the appropriate agents, and preserves data integrity while exchanging JSON-based messages between modules. Since the system relies heavily on AI-generated outputs, testing is not limited to functional correctness but also includes evaluation of response relevance, coherence, and consistency.

The main objectives of testing in this project include:

- Verify correct communication between frontend (React) and backend (FastAPI).
- Validate that each AI agent produces structured and valid JSON outputs.
- Ensure proper orchestration and routing of tasks between multiple agents.
- Test integration with LLM services for consistency, relevance, and reliability of responses.
- Check API handling for latency, failures, and unexpected input scenarios.
- Ensure system stability under concurrent or repeated user requests.
- Validate error handling and fallback mechanisms across the pipeline.

6.2 Types of Testing

Different types of testing are performed to evaluate the functionality and performance of the system.

6.2.1 Black Box Testing

Black Box Testing is a software testing method where the tester checks the functionality of the system without knowing the internal implementation of the code. The tester only focuses on the input and output behavior of the system.

In this project, black box testing is used to verify whether the system correctly processes user queries and returns appropriate responses through the agent orchestration pipeline.

The following aspects are tested using black box testing:

- Checking whether user inputs generate relevant and correct system responses.
- Verifying correct routing of requests to the appropriate agent.
- Ensuring the final output is properly formatted and displayed in the user interface.
- Testing system behavior for invalid or unexpected inputs.

Black box testing ensures that the AgentVerse system works correctly as a complete application from the user's perspective.

6.2.2 White Box Testing

White Box Testing is a testing technique that examines the internal structure, logic, and code implementation of the system. In this method, the tester has knowledge of the internal program structure and verifies whether all logical paths are functioning correctly.

In the AgentVerse system, white box testing is used to test the internal logic of the backend services, agent workflows, and API handling mechanisms.

The following components are tested:

- Backend API request handling (FastAPI endpoints)
- Agent routing and decision-making logic
- JSON parsing and response generation
- LLM integration and response processing

- Error handling and fallback mechanisms
- API data fetching logic

White box testing ensures that all internal modules of the AgentVerse system execute correctly and all logical paths function as expected.

6.2.3 Unit Testing

Unit testing involves testing individual components or modules of the system separately to ensure they function correctly.

In this project, unit testing is performed on each core module independently to verify its correctness before integration with other components.

The following components are tested:

- API request handling module (FastAPI endpoints)
- Agent routing and task assignment logic
- JSON parsing and response generation module
- LLM response processing module
- Error handling and fallback mechanism module

Table 6.2.3 Test Cases (Unit Testing)

S. No	Testcase Description	Actual Output	Expected Output	Result
1.	Start backend server using FastAPI	Server started successfully and API endpoints available	Backend server should run without errors	Pass
2.	Send user query from dashboard interface	Query successfully sent to backend API	System should receive user query correctly	Pass
3.	Detect user intent from query	Intent detected and relevant agents identified	System should correctly identify query intent	Pass

4.	Dynamically create required agents	Agents created based on detected intent	System should generate appropriate agents dynamically	Pass
5.	Execute agents for analysis	Selected agents executed and produced outputs	Agents should analyze query and return results	Pass
6.	Aggregate results from all agents	Aggregated response generated successfully	System should combine outputs into final response	Pass
7.	System should combine outputs into final response	Final response displayed in chat interface	User should see AI response on dashboard	Pass

Each module is tested independently to ensure it produces the expected output for given inputs and handles errors correctly before integrating into the full system.

CHAPTER 7: OUTPUT SCREENS

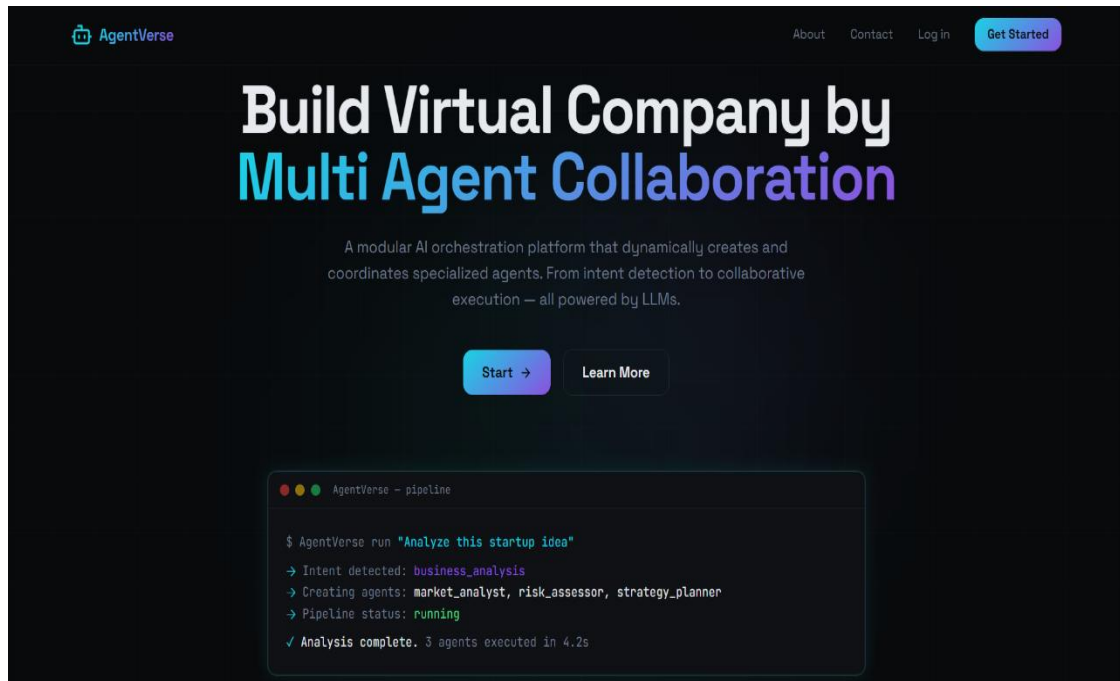


Fig 7.1: Landing Page – 1

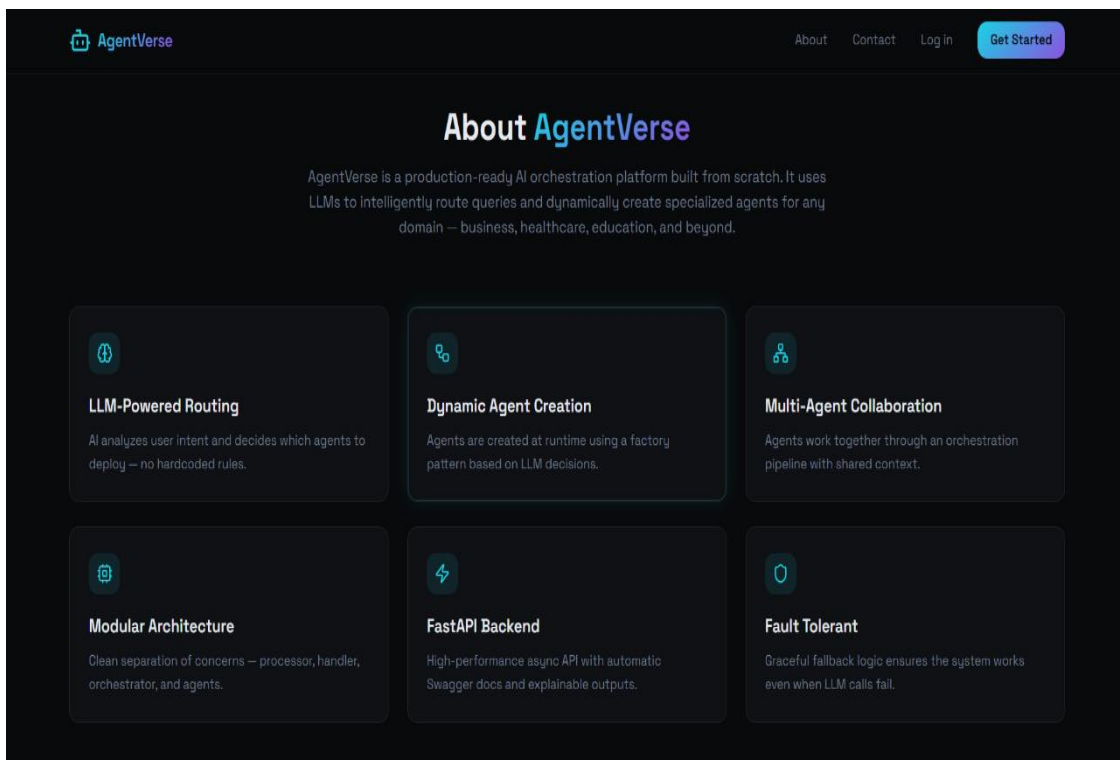


Fig 7.2: Landing Page - 2

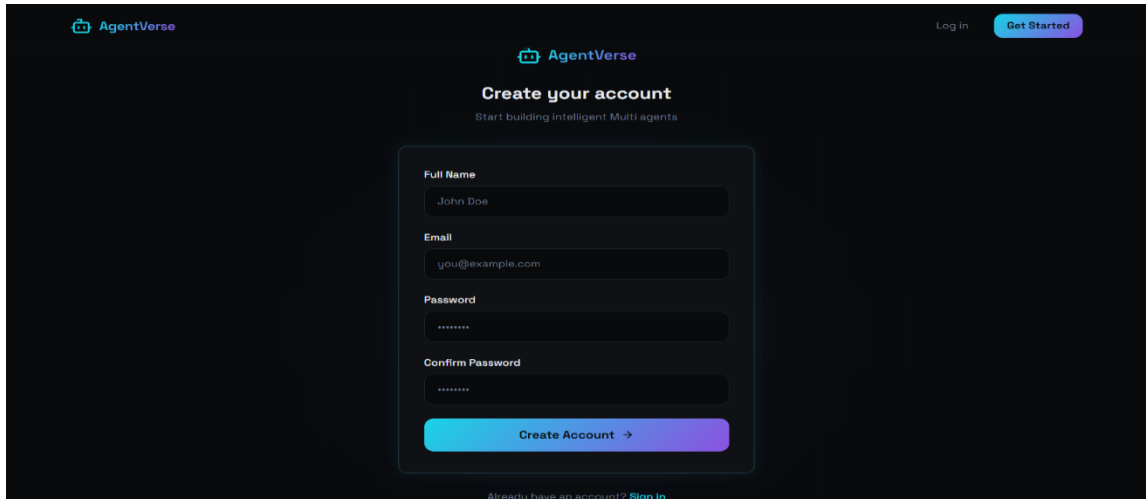


Fig 7.3: Sign Up Page

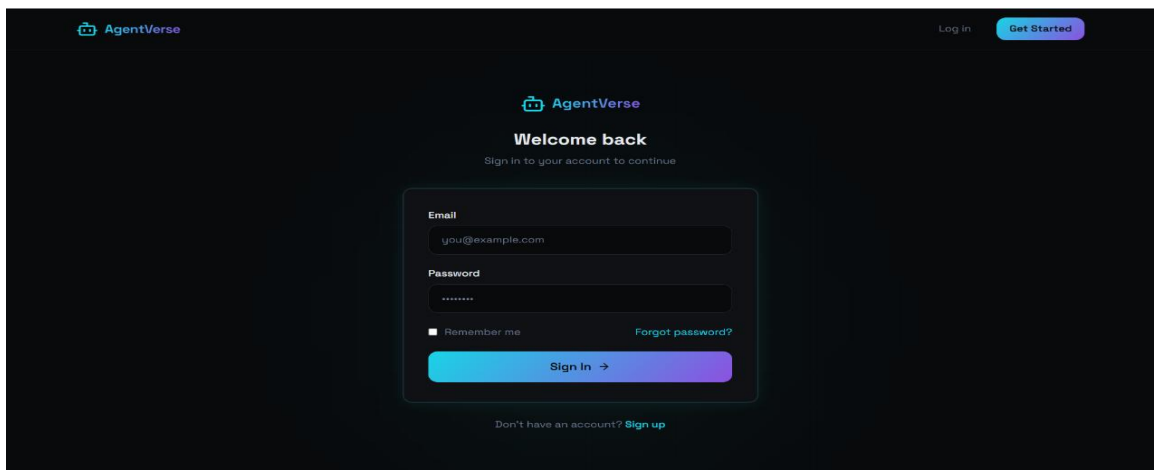


Fig 7.4: Login Page

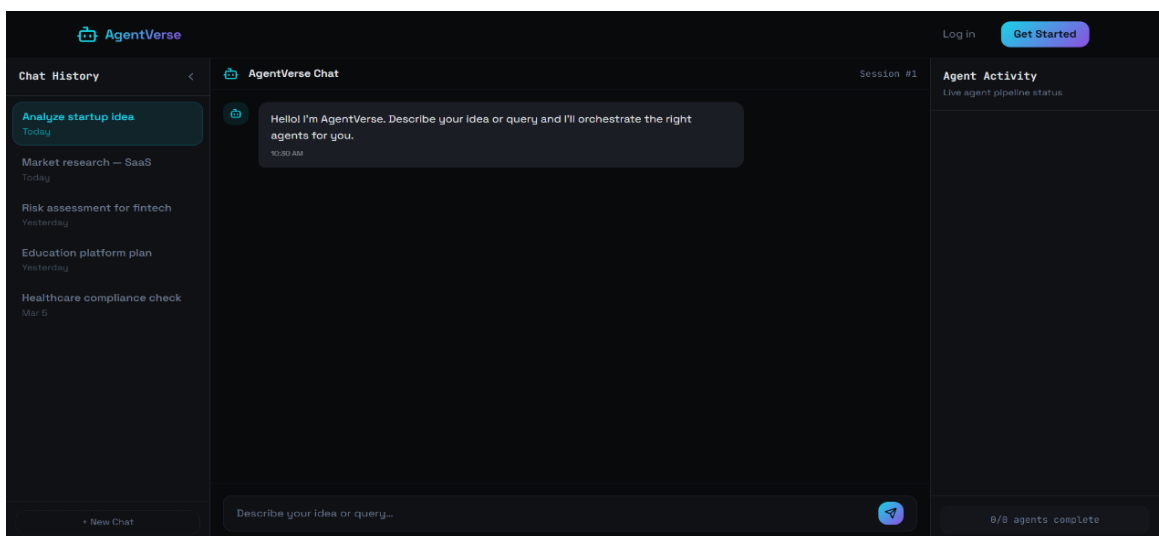


Fig 7.5: AgentVerse Dashboard

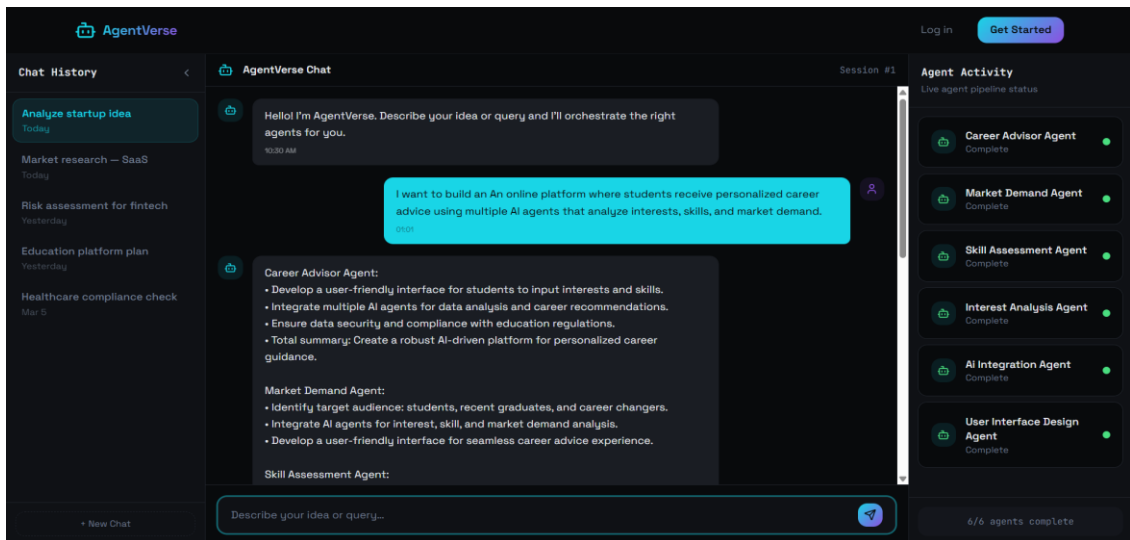


Fig 7.6: Suggestions-1

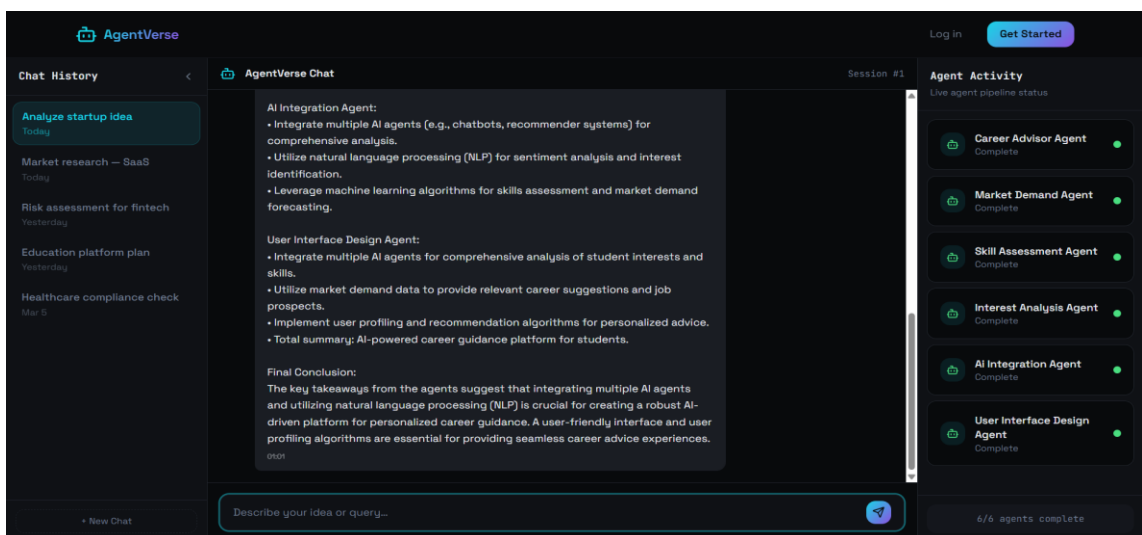


Fig 7.7: Suggestions-2

CONCLUSION

The **AgentVerse system** is developed as a multi-agent AI platform to process user queries intelligently by coordinating multiple specialized agents through an orchestration layer. The system integrates a React-based frontend with a FastAPI backend and leverages LLM-based processing to generate dynamic and context-aware responses

The system enables users to interact with multiple AI agents that handle different tasks such as query understanding, response generation, data formatting, and decision routing. It ensures that user requests are analyzed, assigned to the appropriate agent, and processed to produce structured and meaningful outputs in real time.

The implementation of this system demonstrates how multi-agent architectures and large language models can be combined to build intelligent, scalable, and modular applications. By using JSON-based communication and a structured backend pipeline, the system maintains consistency, flexibility, and ease of integration across components.

Overall, AgentVerse provides a practical example of how AI-driven agent collaboration can be used to build advanced real-world applications that go beyond traditional single-model systems and support more complex user interactions.

Furthermore, the system highlights the importance of modular design and separation of concerns in modern AI applications. Each agent in the system is designed to perform a specific task, which not only improves maintainability but also allows independent updates and scalability. This modular approach makes it easier to extend the system by adding new agents or improving existing ones without affecting the overall architecture.

In addition, the project demonstrates the challenges and solutions involved in integrating LLM-based systems, such as handling inconsistent outputs, ensuring structured responses, and maintaining reliability. By implementing validation mechanisms, fallback strategies, and structured communication formats, the system achieves a balance between flexibility and control, making it more robust and suitable for real-world deployment scenarios.

FUTURE ENHANCEMENTS

Although the AgentVerse system is functioning effectively, there are several improvements that can be implemented in the future to enhance its capabilities and scalability. As the system is built on a multi-agent architecture, it has strong potential to evolve into a more advanced and intelligent platform by incorporating additional features and optimizations.

One major enhancement is the integration of more advanced and specialized AI agents, allowing the system to handle a wider range of complex tasks with higher accuracy and intelligence. This will improve the overall decision-making capability of the orchestration layer.

Another improvement is the incorporation of persistent memory for users, enabling the system to store past interactions and provide more personalized and context-aware responses over time.

The system can also be enhanced by integrating external APIs and real-time data sources, allowing agents to fetch live information and provide more dynamic responses instead of static outputs.

Additionally, improving scalability by deploying the system using containerization tools like Docker and orchestration platforms like Kubernetes can make the system more robust and production-ready.

In addition, future improvements can focus on enhancing the user experience by refining the frontend interface, adding interactive visualizations, and improving response presentation. By combining these enhancements, the AgentVerse system can evolve into a highly intelligent, scalable, and user-friendly platform capable of supporting complex real-world AI applications.

BIBLIOGRAPHY

- [1]. CrewAI Team, “CrewAI: Multi-Agent Orchestration Framework,” (2024) CrewAI Documentation, Available: <https://docs.crewai.com>
- [2]. Ramírez S., “FastAPI: Modern Web Framework for Building APIs with Python,” (2023) FastAPI Documentation, Available: <https://fastapi.tiangolo.com>
- [3]. Pinecone Systems Inc., “Pinecone: Vector Database for Semantic Search,” (2024) Pinecone Documentation, Available: <https://docs.pinecone.io>
- [4]. PostgreSQL Global Development Group, “PostgreSQL: Advanced Open Source Relational Database,” (2023) PostgreSQL Documentation, Available: <https://www.postgresql.org/docs>
- [5]. OpenAI, “OpenAI API: Large Language Models for AI Applications,” (2024) OpenAI Documentation, Available: <https://platform.openai.com/docs>
- [6]. Meta AI, “LLaMA: Open and Efficient Foundation Language Models,” (2023) Research Paper, Available: <https://ai.meta.com>
- [7]. LiteLLM Contributors, “LiteLLM: Unified Interface for LLM Providers,” (2024) LiteLLM Documentation, Available: <https://docs.litellm.ai>
- [8]. Facebook (Meta), “React: A JavaScript Library for Building User Interfaces,” (2023) React Documentation, Available: <https://react.dev>
- [9]. Encode OSS, “Uvicorn: Lightning-Fast ASGI Server Implementation,” (2023) Uvicorn Documentation, Available: <https://www.uvicorn.org>
- [10]. Python Software Foundation, “Python: Programming Language Reference,” (2023) Python Documentation, Available: <https://docs.python.org>
- [11]. Russell S., and Norvig P., “Artificial Intelligence: A Modern Approach,” (2021) 4th Edition, Pearson Education, pp. 1–1152.
- [12]. Kleppmann M., “Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems,” (2017) O’Reilly Media, pp. 1–616.